Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Новосибирский государственный технический университет»

Кафедра _____

Лабораторная работа № ____

по дисциплине «_____»

Тема: «Синтез дискретных А-оптимальных планов эксперимента
для нечетких линейных двухфакторных регрессионных моделей»

Выполнил: студент гр. _____
_____

Проверил: _____

Новосибирск, 2026 г.

1 Задание

Вариант №28. Тема: «Синтез дискретных A-оптимальных планов эксперимента последовательным алгоритмом для нечетких линейных двухфакторных моделей».

Необходимо:

1. Ознакомиться с математическим аппаратом построения регрессионных моделей в рамках концепции нечетких систем и с вопросами оптимального планирования эксперимента.

2. Разработать программное приложение синтеза дискретных A-оптимальных планов эксперимента для двухфакторных нечетких регрессионных моделей. В качестве алгоритма использовать последовательный алгоритм достраивания.

3. Провести численные эксперименты при $\delta \in \{0{,}5; 0{,}4; 0{,}3; 0{,}2\}$ на регулярной сетке 21×21 при максимальном числе точек плана $N_{\max} = 60$; представить таблицы координат точек и графики планов.

2 Теоретическая часть

В задачах идентификации и оптимального планирования эксперимента широко применяются регрессионные модели, описывающие зависимость выходной величины y от нескольких факторов $x_1, \ldots, x_m$. Классические линейные модели предполагают чёткое задание области факторов и единой глобальной зависимости. Однако во многих прикладных задачах (технические системы, социально-экономические объекты и т. п.) естественно использовать *нечеткие регрессионные модели*, основанные на системе нечетких правил вида «если x_1 ..., x_2 ..., то y ...».

В данной работе рассматривается синтез дискретных A-оптимальных планов эксперимента для нечеткой линейной двухфакторной модели. В качестве метода используется последовательный алгоритм достраивания плана на регулярной сетке 21×21 на квадрате $[-1; 1] \times [-1; 1]$. Фаззификация факторов выполняется с помощью двух трапециевидных функций принадлежности для каждого фактора. Исследуется влияние параметра фаззификации δ на характеристики A-оптимальных планов.

2.1 Линейная нечеткая двухфакторная регрессионная модель

2.2 Классическая линейная модель

Классическая двухфакторная линейная регрессионная модель имеет вид

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon, \tag{1}$$

где x_1, x_2 — факторы, $\beta_0, \beta_1, \beta_2$ — неизвестные параметры, ε — случайная ошибка. В матричной форме для вектора наблюдений y записывается

$$y = X\beta + \varepsilon, \tag{2}$$

где X — матрица плана, строки которой содержат значения регрессоров в точках эксперимента.

2.3 Нечеткая регрессионная модель на основе правил

В концепции нечетких систем модель представляется в виде набора правил:

$$R_1 : \text{Если } x_1 \text{ — малый и } x_2 \text{ — малый, то } y = \beta_0^{(11)} + \beta_1^{(11)} x_1 + \beta_2^{(11)} x_2, \tag{3}$$

$$R_2 : \text{Если } x_1 \text{ — малый и } x_2 \text{ — большой, то } y = \beta_0^{(12)} + \beta_1^{(12)} x_1 + \beta_2^{(12)} x_2, \tag{4}$$

$$R_3 : \text{Если } x_1 \text{ — большой и } x_2 \text{ — малый, то } y = \beta_0^{(21)} + \beta_1^{(21)} x_1 + \beta_2^{(21)} x_2, \tag{5}$$

$$R_4 : \text{Если } x_1 \text{ — большой и } x_2 \text{ — большой, то } y = \beta_0^{(22)} + \beta_1^{(22)} x_1 + \beta_2^{(22)} x_2. \tag{6}$$

Каждое правило определяет *локальную* линейную модель в определённой области пространства факторов. Степень влияния правила при конкретных значениях x_1, x_2 определяется величинами степеней принадлежности этих значений соответствующим нечетким термам.

Пусть для фактора x_1 заданы две нечеткие партиции с функциями принадлежности $\mu_1(x_1)$ и $\mu_2(x_1)$, аналогично для x_2 — $\mu_1(x_2)$ и $\mu_2(x_2)$. Тогда веса правил вычисляются как

$$w_{11} = \mu_1(x_1)\mu_1(x_2), \tag{7}$$

$$w_{12} = \mu_1(x_1)\mu_2(x_2), \tag{8}$$

$$w_{21} = \mu_2(x_1)\mu_1(x_2), \tag{9}$$

$$w_{22} = \mu_2(x_1)\mu_2(x_2). \tag{10}$$

Итоговый выход модели определяется агрегированием по всем правилам:

$$y(x_1, x_2) = \sum_{i=1}^{2} \sum_{j=1}^{2} w_{ij}(x_1, x_2) \left(\beta_0^{(ij)} + \beta_1^{(ij)} x_1 + \beta_2^{(ij)} x_2 \right). \tag{11}$$

Таким образом, модель остаётся линейной по параметрам $\beta_k^{(ij)}$, но учитывает нелинейный характер влияния факторов за счёт нечеткого разбиения области определения.

2.4 Фаззификация и трапециевидные нечеткие партиции

2.5 Понятие фаззификации

Фаззификация (fuzzification) — это переход от чётких значений входных величин к их нечеткому представлению. Каждому значению x сопоставляется вектор степеней принадлежности к нескольким нечетким термам:

$$x \mapsto \big(\mu_1(x), \mu_2(x), \ldots, \mu_K(x)\big), \quad \mu_k(x) \in [0, 1]. \tag{12}$$

При этом одно и то же значение может частично принадлежать сразу нескольким термам (например, «немного малый» и «немного большой»).

2.6 Трапециевидные функции принадлежности

В задании область определения каждого фактора x_i равна отрезку $[-1; 1]$, который разбивается на две трапециевидные нечеткие партиции с функциями принадлежности $\mu_1(x)$ и $\mu_2(x)$ следующего вида:

$$\mu_1(x) = \begin{cases} 1, & x \le -\delta, \\ \dfrac{\delta - x}{2\delta}, & -\delta \le x \le \delta, \\ 0, & x > \delta, \end{cases} \qquad \mu_2(x) = 1 - \mu_1(x), \tag{13}$$

где $\delta > 0$ — параметр ширины переходной области. Для малых $|x|$ значения μ_1 и μ_2 плавно изменяются от 1 до 0, обеспечивая «мягкое» разделение области факторов на две партиции.

В работе рассматриваются значения

$$\delta \in \{0{,}5;\ 0{,}4;\ 0{,}3;\ 0{,}2\},$$

что позволяет исследовать влияние степени «размытия» разбиения на свойства A-оптимального плана.

2.7 Информационная матрица и критерий A-оптимальности

2.8 Вектор регрессоров

Для каждой точки плана (x_1, x_2) формируется 12-мерный вектор регрессоров, соответствующий четырём нечетким правилам и трём регрессорам в каждом правиле:

$$f(x_1, x_2) = \left[w_{11}, w_{11}x_1, w_{11}x_2,\ w_{12}, w_{12}x_1, w_{12}x_2,\ w_{21}, w_{21}x_1, w_{21}x_2,\ w_{22}, w_{22}x_1, w_{22}x_2\right]^T. \quad (14)$$

Всего в модели $4 \times 3 = 12$ параметров.

2.9 Информационная матрица

Пусть план эксперимента состоит из набора точек

$$\xi = \{(x_1^{(k)}, x_2^{(k)})\}_{k=1}^{N}, \quad N \le 60.$$

Информационная матрица модели определяется как

$$M(\xi) = \sum_{k=1}^{N} f(x_1^{(k)}, x_2^{(k)})\, f(x_1^{(k)}, x_2^{(k)})^T. \quad (15)$$

Размерность M равна 12×12. При предположении о нормальности ошибок оценка дисперсионной матрицы вектора оценок параметров пропорциональна M^{-1}.

2.10 Критерий A-оптимальности

Критерий A-оптимальности направлен на минимизацию суммарной дисперсии оценок параметров и определяется как

$$\Phi_A(\xi) = \operatorname{tr}\left(M(\xi)^{-1}\right). \quad (16)$$

В программной реализации используется схема, аналогичная ЛР3:

$$A_s = \begin{cases} M_s + \gamma I, & s < m, \\ M_s, & s \ge m, \end{cases} \qquad \Phi_A(\xi_s) = \operatorname{tr}\left(A_s^{-1}\right), \quad (17)$$

где $m = 12$ — число параметров модели, $\gamma = 10^{-8}$. Такая регуляризация применяется только на ранних шагах, когда информационная матрица ещё вырождена.

2.11 Последовательный алгоритм достраивания плана

Для синтеза дискретного A-оптимального плана используется последовательный алгоритм достраивания:

1. Задаётся дискретная область планирования — регулярная сетка 21×21 на квадрате $[-1;1] \times [-1;1]$. Максимальное число точек плана $N_{\max} = 60$.

2. Начинаем с пустого плана: $M_0 = 0,\ \xi_0 = \varnothing$.

3. На t-м шаге ($t = 0, 1, \ldots, N_{\max} - 1$) для каждой точки (x_1, x_2) сетки вычисляется *кандидатная* информационная матрица

$$M_{\text{cand}} = M_t + f(x_1, x_2) f(x_1, x_2)^T \tag{18}$$

   и значение A-критерия $\Phi_A(M_{\text{cand}})$.

4. Выбирается точка, дающая минимальное значение критерия:

$$(x_1^*, x_2^*) = \arg \min_{(x_1,x_2)\in G} \Phi_A\big(M_t + f f^T\big), \tag{19}$$

   где G — множество всех узлов сетки.

5. На шагах $t < m$ для обеспечения невырожденности матрицы запрещаются повторы и отбрасываются кандидаты, не увеличивающие ранг M_t. Начиная с $t \geq m$ повторные наблюдения допускаются.

6. Обновляются матрица и план:

$$M_{t+1} = M_t + f(x_1^*, x_2^*) f(x_1^*, x_2^*)^T, \quad \xi_{t+1} = \xi_t \cup \{(x_1^*, x_2^*)\}. \tag{20}$$

7. Процесс повторяется до достижения $|\xi_t| = N_{\max} = 60$.

Алгоритм реализует жадное достраивание плана и является стандартным подходом к построению дискретных A-оптимальных планов (см., например, работы А. А. Попова по оптимальному планированию эксперимента для нечетких регрессионных моделей).

3 Результаты численных экспериментов

3.1 Тестовые примеры

В соответствии с заданием проведены эксперименты для

$$\delta \in \{0{,}5;\ 0{,}4;\ 0{,}3;\ 0{,}2\},$$

область факторов $x_1, x_2 \in [-1;1]$, область планирования — регулярная сетка 21×21, максимальное число точек в плане $N_{\max} = 60$.

Для каждого значения δ был синтезирован дискретный A-оптимальный план из 60 точек и вычислены характеристики информационной матрицы M: след A-обратной матрицы, детерминант, ранг и число обусловленности.

Таблица 1: Характеристики информационной матрицы M для различных δ

δ	n_{uniq}	$\text{tr}(M^{-1})$	$\det(M)$	ранг	$\text{cond}(M)$
0.5	35	$1.254265 \cdot 10^1$	$3.264057 \cdot 10^4$	12	$7.690982 \cdot 10^1$
0.4	31	$8.879718 \cdot 10^0$	$3.271588 \cdot 10^5$	12	$4.562219 \cdot 10^1$
0.3	27	$6.540337 \cdot 10^0$	$2.887068 \cdot 10^6$	12	$2.748598 \cdot 10^1$
0.2	24	$4.954718 \cdot 10^0$	$2.363074 \cdot 10^7$	12	$1.796948 \cdot 10^1$

3.2 Численные характеристики матрицы M

Сводная таблица результатов:

Из таблицы видно, что при уменьшении параметра фаззификации δ значение критерия A-оптимальности $\text{tr}(M^{-1})$ монотонно уменьшается, а детерминант матрицы M существенно возрастает. Число обусловленности $\text{cond}(M)$ также уменьшается, что свидетельствует о лучшей численной устойчивости оценки параметров. Во всех случаях ранг матрицы равен 12, а число уникальных точек спектра уменьшается при уменьшении δ (растут кратности наблюдений).

3.3 Графическое расположение точек планов

На рисунках 1–4 представлены графики расположения точек A-оптимальных планов для различных значений параметра δ. Каждый план содержит 60 наблюдений на квадрате $[-1; 1] \times [-1; 1]$. Размер маркера на рисунке пропорционален кратности точки спектра.

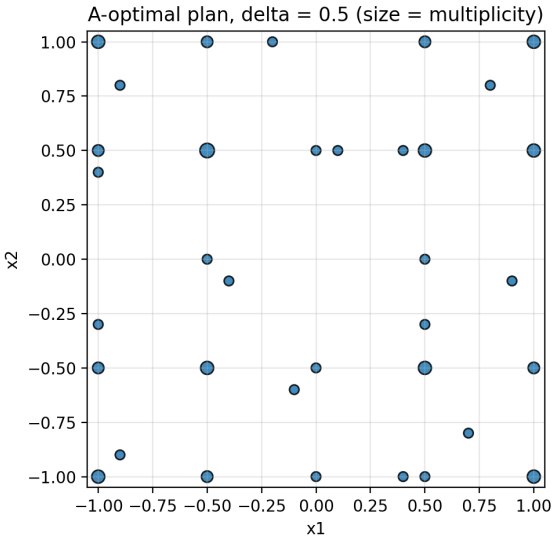

Рис. 1: A-оптимальный план при $\delta = 0{,}5$

Визуальный анализ показывает, что точки планов концентрируются вблизи границ и в окрестности нуля, обеспечивая хороший баланс между глобальным покрытием области факторов и точностью оценивания параметров в наиболее информативных зонах.

3.4 Координаты точек и спектры планов

Для каждого δ программа выводит две таблицы:

1. список из 60 наблюдений (координаты точек, включая повторы);

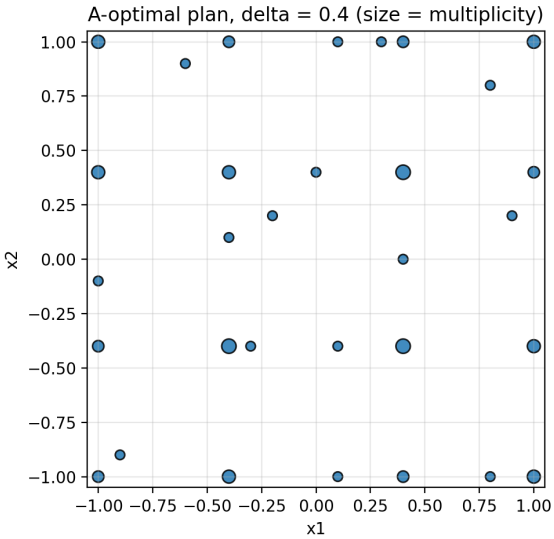

Рис. 2: A-оптимальный план при $\delta = 0{,}4$

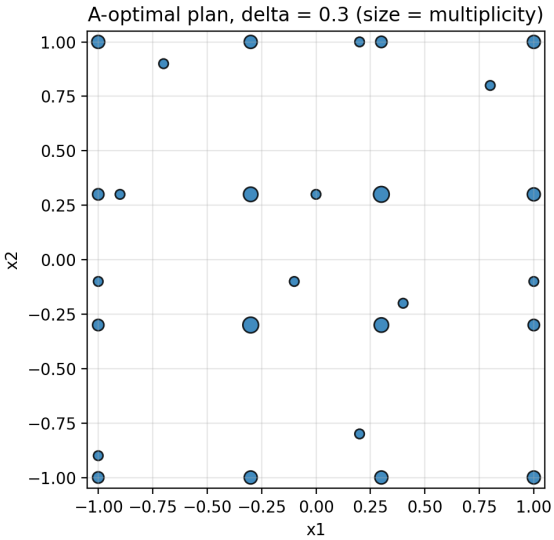

Рис. 3: A-оптимальный план при $\delta = 0{,}3$

2. спектр плана: уникальные точки с кратностями r_i и весами $p_i = r_i/60$.

Полные таблицы занимают большой объём и формируются автоматически в протоколе запуска `/tmp/model_run_full.txt`.

4 Программная реализация

Приложение реализовано на языке `Python` с использованием библиотек `NumPy` и `Matplotlib`. Структура исходного кода:

- Файл `src/fuzzy.py`: функции фаззификации и формирования регрессора.
- Файл `src/model.py`: генерация сетки, вычисление информационной матрицы, реализация алгоритма синтеза, расчёт характеристик матрицы, построение графиков и вывод таблиц.

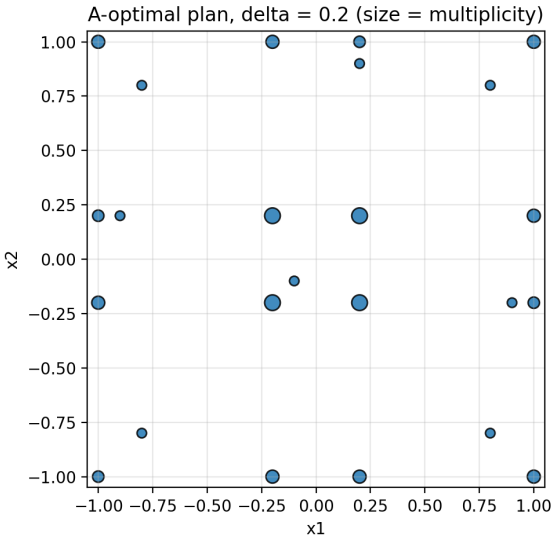

Рис. 4: A-оптимальный план при $\delta = 0{,}2$

Основные функции:

- `mu1(x, delta)`, `mu2(x, delta)` — трапециевидные функции принадлежности для фаззификации факторов.
- `fuzzy_weights(x1, x2, delta)` — вычисление весов правил $w_{11}, w_{12}, w_{21}, w_{22}$.
- `regressor_vector(x1, x2, delta)` — формирование 12-мерного вектора регрессоров $f(x)$.
- `compute_info_matrix(points, delta)` — вычисление информационной матрицы M для заданного набора точек.
- `synthesize_a_optimal_plan(delta, grid, max_points, gamma)` — реализация последовательного алгоритма достраивания с условной регуляризацией (как в ЛР3).
- `build_spectrum(points)` — построение спектра плана: (x_1, x_2, r_i, p_i).
- `analyze_matrix(M, num_points, gamma)` — расчёт $\mathrm{tr}(M^{-1})$, $\det(M)$, ранга и числа обусловленности.
- `plot_plan(points, delta)` — построение и сохранение графика расположения точек плана с учётом кратностей.
- `run_synthesis_for_deltas(log_every=10)` — запуск экспериментов для $\delta \in \{0{,}5; 0{,}4; 0{,}3; 0{,}2\}$.

Запуск приложения осуществляется командой

```
python -m src.model
```

из корневой директории проекта. По завершении работы для каждого значения δ выводятся таблицы координат точек плана и численные характеристики матрицы M, а также сохраняются рисунки с расположением точек в файлы `plan_delta_0.50.png`, `plan_delta_0.40.png`, `plan_delta_0.30.png`, `plan_delta_0.20.png`.

Заключение

В работе разработано и реализовано программное приложение для синтеза дискретных A-оптимальных планов эксперимента для нечеткой линейной двухфакторной регрессионной модели на основе последовательного алгоритма достраивания плана на регулярной сетке.

Проведённые численные эксперименты для значений $\delta \in \{0{,}5;\ 0{,}4;\ 0{,}3;\ 0{,}2\}$ показывают, что при уменьшении параметра фаззификации улучшаются характеристики A-оптимального плана: уменьшается значение $\mathrm{tr}(M^{-1})$, возрастает детерминант M и снижается число обусловленности. При этом получаемые планы имеют выраженную дискретную структуру спектра с повторными наблюдениями, что соответствует постановке задачи дискретного оптимального планирования.

Текст программы

Файл `src/fuzzy.py`

```
# src/fuzzy.py

import numpy as np


def _validate_delta(delta: float) -> None:
    if delta <= 0:
        raise ValueError("delta must be positive")


def mu1(x: float, delta: float) -> float:
    """Left trapezoidal membership function."""
    _validate_delta(delta)
    if x <= -delta:
        return 1.0
    if x >= delta:
        return 0.0
    return (delta - x) / (2 * delta)


def mu2(x: float, delta: float) -> float:
    """Right trapezoidal membership function."""
    return 1.0 - mu1(x, delta)


def fuzzy_weights(x1: float, x2: float, delta: float) -> tuple[float, float, float, floa
    """Rule weights w_ij for (x1, x2)."""
    m1 = mu1(x1, delta)
    m2 = mu2(x1, delta)
    n1 = mu1(x2, delta)
    n2 = mu2(x2, delta)

    w11 = m1 * n1
    w12 = m1 * n2
```

```
    w21 = m2 * n1
    w22 = m2 * n2
    return w11, w12, w21, w22


def regressor_vector(x1: float, x2: float, delta: float) -> np.ndarray:
    """Regressor vector (size 12) for a two-factor fuzzy linear model."""
    w11, w12, w21, w22 = fuzzy_weights(x1, x2, delta)
    return np.array(
        [
            w11,
            w11 * x1,
            w11 * x2,
            w12,
            w12 * x1,
            w12 * x2,
            w21,
            w21 * x1,
            w21 * x2,
            w22,
            w22 * x1,
            w22 * x2,
        ],
        dtype=float,
    )


def basis_vector(x1: float, x2: float, delta: float) -> np.ndarray:
    """Backward-compatibility alias."""
    return regressor_vector(x1, x2, delta)
```

Файл src/model.py

```
# src/model.py

import numpy as np
import matplotlib.pyplot as plt

from src.fuzzy import regressor_vector


def generate_grid(n: int = 21) -> list[tuple[float, float]]:
    """Regular n x n grid on [-1, 1] x [-1, 1]."""
    pts = np.linspace(-1, 1, n)
    return [(float(x1), float(x2)) for x1 in pts for x2 in pts]


def compute_info_matrix(points: list[tuple[float, float]], delta: float) -> np.ndarray:
    """Information matrix M = sum f(x) f(x)^T for selected points."""
    dim = 12
    M = np.zeros((dim, dim), dtype=float)
```

```
    for x1, x2 in points:
        f = regressor_vector(x1, x2, delta)
        M += np.outer(f, f)
    return M


def a_criterion(M: np.ndarray, num_points: int, gamma: float = 1e-8) -> float:
    """A-optimality criterion in the style of lab #3.

    If s < m, use M + gamma*I.
    If s >= m, use plain M.
    """
    dim = M.shape[0]
    A = M + gamma * np.eye(dim) if num_points < dim else M
    try:
        invA = np.linalg.inv(A)
    except np.linalg.LinAlgError:
        invA = np.linalg.pinv(A)
    return float(np.trace(invA))


def synthesize_a_optimal_plan(
    delta: float,
    grid: list[tuple[float, float]],
    max_points: int = 60,
    gamma: float = 1e-8,
    log_every: int = 0,
) -> tuple[list[tuple[float, float]], np.ndarray]:
    """Sequential A-optimal plan construction.

    Multiplicities are allowed in a discrete plan.
    """
    num_candidates = len(grid)
    dim = 12

    FFt = np.zeros((num_candidates, dim, dim), dtype=float)
    for i, (x1, x2) in enumerate(grid):
        f = regressor_vector(x1, x2, delta)
        FFt[i, :, :] = np.outer(f, f)

    selected_indices: list[int] = []
    selected_once = np.zeros(num_candidates, dtype=bool)
    M = np.zeros((dim, dim), dtype=float)

    for step in range(max_points):
        best_idx = -1
        best_value = np.inf
        rank_M = np.linalg.matrix_rank(M)

        for idx in range(num_candidates):
```

```
            # As in lab #3: no repeats before first m points.
            if step < dim and selected_once[idx]:
                continue
            Mcand = M + FFt[idx]
            # As in lab #3: enforce rank growth on early steps.
            if step < dim and np.linalg.matrix_rank(Mcand) <= rank_M:
                continue
            value = a_criterion(Mcand, num_points=step + 1, gamma=gamma)
            if value < best_value:
                best_value = value
                best_idx = idx

        if best_idx < 0:
            break

        selected_indices.append(best_idx)
        selected_once[best_idx] = True
        M += FFt[best_idx]

        if log_every > 0 and (step == 0 or (step + 1) % log_every == 0 or (step + 1) ==
            cur_val = a_criterion(M, num_points=step + 1, gamma=gamma)
            uniq = len(set(selected_indices))
            print(f"{step + 1:3d}\t{cur_val: .6e}\t{uniq:3d}")

    plan_points = [grid[i] for i in selected_indices]
    return plan_points, M


def build_spectrum(
    points: list[tuple[float, float]],
) -> list[tuple[float, float, int, float]]:
    """Build plan spectrum: unique points, counts r_i and weights p_i."""
    if not points:
        return []

    counts: dict[tuple[float, float], int] = {}
    for p in points:
        counts[p] = counts.get(p, 0) + 1

    total = len(points)
    spectrum = [(x1, x2, r, r / total) for (x1, x2), r in counts.items()]
    spectrum.sort(key=lambda row: (-row[2], row[0], row[1]))
    return spectrum


def analyze_matrix(M: np.ndarray, num_points: int, gamma: float = 1e-8) -> dict:
    """Compute matrix metrics."""
    dim = M.shape[0]
    A = M + gamma * np.eye(dim) if num_points < dim else M
    try:
```

```
        invA = np.linalg.inv(A)
    except np.linalg.LinAlgError:
        invA = np.linalg.pinv(A)

    trace_inv = float(np.trace(invA))
    det = float(np.linalg.det(M))
    rank = int(np.linalg.matrix_rank(M))
    try:
        cond = float(np.linalg.cond(M))
    except np.linalg.LinAlgError:
        cond = float("inf")

    return {
        "trace_inv": trace_inv,
        "det": det,
        "rank": rank,
        "cond": cond,
    }


def plot_plan(points: list[tuple[float, float]], delta: float, save: bool = True) -> Non
    """Plot plan points on [-1, 1] x [-1, 1]."""
    if not points:
        return
    spectrum = build_spectrum(points)
    xs = [row[0] for row in spectrum]
    ys = [row[1] for row in spectrum]
    rs = np.array([row[2] for row in spectrum], dtype=float)
    sizes = 35 + 15 * (rs - 1)

    plt.figure(figsize=(5, 5))
    plt.scatter(xs, ys, c="tab:blue", s=sizes, edgecolors="k", alpha=0.85)
    plt.xlim(-1.05, 1.05)
    plt.ylim(-1.05, 1.05)
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.title(f"A-optimal plan, delta = {delta} (size = multiplicity)")
    plt.gca().set_aspect("equal", adjustable="box")
    plt.grid(True, alpha=0.3)
    plt.tight_layout()

    if save:
        filename = f"plan_delta_{delta:.2f}.png"
        plt.savefig(filename, dpi=200)
        plt.close()


def print_plan_table(points: list[tuple[float, float]]) -> None:
    """Print full point list (with repeats)."""
    print("No\tx1\t\tx2")
```

```
    for i, (x1, x2) in enumerate(points, start=1):
        print(f"{i:2d}\t{x1: .3f}\t{x2: .3f}")


def print_spectrum_table(points: list[tuple[float, float]]) -> None:
    """Print spectrum: unique points, counts and weights."""
    spectrum = build_spectrum(points)
    print("No\tx1\t\tx2\tr_i\tp_i")
    for i, (x1, x2, r, p) in enumerate(spectrum, start=1):
        print(f"{i:2d}\t{x1: .3f}\t{x2: .3f}\t{r:2d}\t{p:.4f}")


def run_synthesis_for_deltas(
    deltas: list[float] | tuple[float, ...] = (0.5, 0.4, 0.3, 0.2),
    max_points: int = 60,
    gamma: float = 1e-8,
    log_every: int = 0,
) -> None:
    """Run plan synthesis for selected deltas."""
    grid = generate_grid(n=21)
    summary_rows: list[tuple[float, int, float, float, int, float]] = []

    for delta in deltas:
        print(f"\n===== delta = {delta} =====")
        if log_every > 0:
            print("Iter\ttrace(A^-1)\tuniq")
        plan_points, M = synthesize_a_optimal_plan(
            delta,
            grid,
            max_points=max_points,
            gamma=gamma,
            log_every=log_every,
        )
        stats = analyze_matrix(M, num_points=len(plan_points), gamma=gamma)
        uniq = len(build_spectrum(plan_points))
        summary_rows.append((delta, uniq, stats["trace_inv"], stats["det"], stats["rank"

        print("\nPlan points (with repeats):")
        print_plan_table(plan_points)
        print("\nSpectrum (unique points, multiplicities, weights):")
        print_spectrum_table(plan_points)

        print("\nInfo matrix M metrics:")
        print(f"trace(M^(-1)) ~= {stats['trace_inv']:.6e}")
        print(f"det(M)        ~= {stats['det']:.6e}")
        print(f"rank(M)       = {stats['rank']}")
        print(f"cond(M)       ~= {stats['cond']:.6e}")

        plot_plan(plan_points, delta, save=True)
```

```
    print("\n===== Summary by delta =====")
    print("delta\tuniq\ttrace(M^-1)\t\tdet(M)\t\trank\tcond(M)")
    for delta, uniq, trace_inv, det, rank, cond in summary_rows:
        print(f"{delta:.1f}\t{uniq:2d}\t{trace_inv: .6e}\t{det: .6e}\t{rank:2d}\t{cond:


if __name__ == "__main__":
    run_synthesis_for_deltas(log_every=10)
```

Литература

1. Попов А. А. Регрессионное моделирование на основе нечетких правил // Сборник научных трудов НГТУ. 2000. № 2(19). С. 49–57.

2. Пегат А. Нечеткое моделирование и управление. 2-е изд. М.: БИНОМ. Лаборатория знаний, 2013. 798 с.

3. Попов А. А. Планирование и анализ эксперимента. Конспект лекций. Электронный вариант.

4. Попов А. А. Оптимальное планирование эксперимента в задачах структурной и параметрической идентификации моделей многофакторных систем: монография. Новосибирск: Изд-во НГТУ, 2013. 296 с.

5. Попов А. А. Оптимальное планирование эксперимента при активной идентификации нечетких линейных регрессионных моделей // Modeling, Optimization and Information Technology. 2019. Т. 7, № 1.

6. Халов Е. Л. Систематический обзор чётких одномерных функций принадлежности интеллектуальных систем.